GUIDE 45 · GOVERN

Hide personal data from a partner or an AI agent

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=45-hide-personal-data-from-a-partner-or-an-ai-agent. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** one saved query - or a connection a partner or an AI agent may write SQL against - where the email addresses come back as stable codes and names as `I**`, whatever the SQL calls the columns; queries that masking can't follow refused rather than served; and a record of what was masked, for whom.

Every command and response below was run against a real server with the example APIs loaded (`demo-key` is an admin).

Step 1: Give the caller its rules

Masking rules most often belong to the **caller** - an API key, a role (and so the keys made from it and the signed-in users mapped to it), or an export - so one saved query serves your own team as it is and a partner masked. A rule can also belong to the **query itself**, when a column should be masked for *everyone* who runs it, admins included - see Step 6.

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "acme-partner", "connections": ["examples"],
  "mask": [{"column": "customer.email", "action": "hash"},
           {"column": "customer.last_name", "action": "mask"}]
}'
```

A rule names a **source column** - `table.column`, or `schema.table.column`, with `*` as a wildcard (`*.email`, `customer.*_name`) - and what to do with its values:

ACTION	TURNS `ISLA.ITO1@EXAMPLE.COM` INTO	WHEN
`hash`	`eafbbd68098fdc3c`	the same value always gives the same code, so counting, grouping and joining still work
`mask`	`i**@example.com` (text: `Ito` -> `I**`)	people should see there is a value, not what it is
`null`	`null`	the column should stay, empty
`drop`	*(column removed)*	it shouldn't appear at all
`keep`	*(unchanged)*	to say a column is fine as it is - see views, below

The strictest rule that applies wins (`drop` > `null` > `mask` > `hash`). In the Console: **API keys → New API key → Column masking**; keys and roles with rules carry a **masked** tag, in their tables and on the Access map.

Step 2: See it

The same saved query, for the admin and for the partner:

```shell
curl http://127.0.0.1:5000/q/customer_contacts -H 'X-API-Key: demo-key'
# [{"customer_id": 1, "first_name": "Isla", "last_name": "Ito", "contact": "isla.ito1@example.com"},
...]

curl http://127.0.0.1:5000/q/customer_contacts -H 'X-API-Key: <acme-partner>'
# [{"customer_id": 1, "first_name": "Isla", "last_name": "I***", "contact": "eafbbd68098fdc3c"},
#  {"customer_id": 2, "first_name": "Ava", "last_name": "I***", "contact": "4b3ca6ce45e20366"}, ...]
```

The query calls the column `contact`, not `email` - it's still hashed, because rules follow the data, not the name. The same goes for every format (`?format=csv`, Parquet), `?stream=true` exports, MCP tools and the cache: a partner is never served a response cached for someone else.

The caller can see what it gets masked, in `GET /catalog`:

```json
{"name": "customer_contacts",
 "masking": {"masked": [{"column": "last_name", "action": "mask"}, {"column": "contact", "action":
"hash"}],
             "refused": null}}
```

Step 3: Let them write SQL - masking holds

A partner, or an AI agent over MCP, with the connection in its grant can write its own SQL. Each result column is traced through the SQL to the table columns it's computed from, so renaming, wrapping or nesting doesn't get the value out:

```shell
# SELECT email FROM customer LIMIT 2
[{"email": "eafbbd68098fdc3c"}, {"email": "4b3ca6ce45e20366"}]
# SELECT upper(email) AS e FROM customer LIMIT 2           - computed from email: hashed as a whole
[{"e": "f94742444fd906a6"}, {"e": "b313c006e2be7cad"}]
# WITH x AS (SELECT email AS e FROM customer) SELECT e AS anything FROM x LIMIT 2
[{"anything": "eafbbd68098fdc3c"}, {"anything": "4b3ca6ce45e20366"}]
```

A query masking can't follow is refused - before it runs - rather than served unmasked:

```shell
# SELECT c FROM customer c        - the whole row as one value
{"error": "This query's result columns can't be traced to their sources for masking: Column '\"c\"'
could not be
 resolved", "code": "mask_unresolvable"}
```

Refused the same way: table functions (`read_parquet(...)`), a table alias that renames columns (`customer AS c(a, e)`), `SELECT *` over a `NATURAL` or `USING` join, `LATERAL`, recursive CTEs, DuckDB's `COLUMNS('...')`, and functions masking can't see into (`version()`) - name the columns instead. A caller with rules also never sees the database's own error text, which can quote a value.

Step 4: Know it happened

Every run records what was masked, in run history:

```json
{"key_name": "acme-partner", "masked": [{"column": "email", "action": "hash", "source":
"customer.email"}], ...}
```

So "was the email ever returned unmasked to this partner?" is a search in `GET /api/v1/history?key=acme-partner`.

Step 5: Files for the partner, masked too

An export carries its own rules, whoever runs it (deliver a daily file):

```shell
curl -X POST http://127.0.0.1:5000/api/v1/exports -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "acme-contacts", "query": "customer_contacts", "destination": "acme-drop", "format": "csv",
  "path": "contacts/{date}.csv",
  "mask": [{"column": "customer.email", "action": "hash"}, {"column": "customer.last_name", "action":
"drop"}]
}'
```

```text
customer_id,first_name,contact
1,Isla,eafbbd68098fdc3c
2,Ava,4b3ca6ce45e20366
```

The codes match the partner's API key's: one hash key per store, so a partner can join a file to an API response on them. Masking an export's incremental column is refused - the watermark is made of its values.

Step 6: Or mask a column for everyone, on the query

When a column should never leave a particular query unmasked - no matter who calls it, admins included - put the rule on the **query** rather than on each caller. In the Console it's the **Column masking** field on the save form (API Repository → New version, or the API Designer's **Save as New API**); the same field in the Designer's **Settings** pane masks a preview before you save. In a query definition it's a `mask` array, the same shape as a caller's:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "customer_contacts", "connection_name": "examples",
  "sql": "SELECT customer_id, first_name, last_name, email AS contact FROM customer",
  "mask": [{"column": "customer.email", "action": "hash"}]
}'
```

Now *everyone* who runs `customer_contacts` gets `contact` hashed, including `demo-key`. A caller that also carries a mask on its key gets **both**: per column, the stricter action wins (`drop` > `null` > `mask` > `hash`), so a query's `hash` and a key's `drop` on the same column drops it. Query-level masking traces SQL columns the same way, so it's not

available on Mongo queries, and a query whose mask can't be traced is refused rather than served unmasked - exactly as for a caller's rules.

What masking doesn't stop

Asking about a value without selecting it. A caller that writes SQL can still filter and group on a masked column:

```shell
# SELECT count(*) AS n FROM customer WHERE email LIKE 'a%'
[{"n": 15}]
```

Masking hides values in what comes back; it doesn't stop a question about them. When that matters - an AI agent near sensitive data - give the caller **saved queries only** (no `connections` grant), or `allowed_tables` on a view that leaves the column out. Hashes are keyed, so a likely value can't be hashed and compared - but equal values are visible as equal, which is the point of `hash`.

Good to know

- **Views** are traced only as far as the view. A rule on `customer.email` can't tell whether `contact_view.email` is that column, so a query reading it is refused until a rule names the view: `*.email` (which covers every table and view), `contact_view.email` with an action, or `contact_view.email` with `keep` to say it's fine.
- **Window functions** are masked when they order or partition by a masked column - `RANK() OVER (ORDER BY email)` comes back hashed. Over other columns they're fine.
- **Administrators see data as it is** - masking is for the callers you hand data to. Auditors see no data at all.
- **The hash key** is made once per store and shared by every instance; set `QUERYAPIGATE_MASK_KEY` to choose it. Changing it changes every code.
- **Supported** on MySQL, PostgreSQL, ClickHouse, SQLite and DuckDB connections (it needs the `sqlglot` package - `queryapigate[flow]`, in the Docker image). A caller with rules is refused on H2, JDBC and MongoDB connections, for now.

Next steps

- Let an AI agent call your saved queries via MCP - with a masked key.
- Restrict a key to specific tables - for columns that shouldn't be queryable at all.
- Deliver a daily file to a partner's bucket.